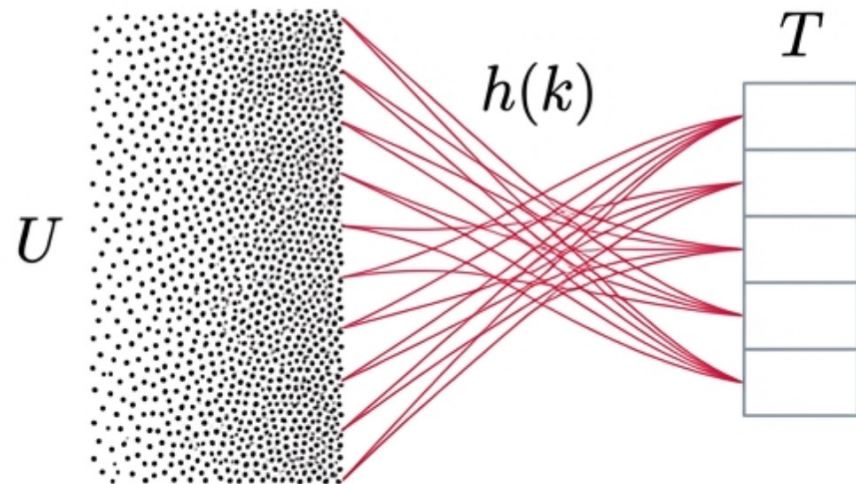
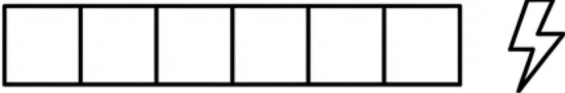
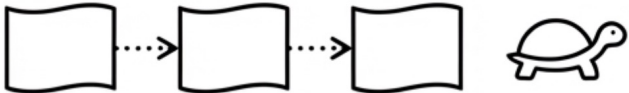




Tablas hash



El dilema

 Array	 Linked List
Tiempo de Acceso: Constante $O(1)$	Tiempo de Acceso: Variable $O(n)$
Eficiencia de Espacio: Tamaño Fijo	Eficiencia de Espacio: Tamaño Dinámico
Adaptabilidad: Limitada	Adaptabilidad: Flexible
Inserción/Eliminación: Requiere redimensionamiento	Inserción/Eliminación: Sin redimensionamiento
Sobrecarga de Memoria: Mínima	Sobrecarga de Memoria: Ligeramente mayor (punteros)

Tablas hash

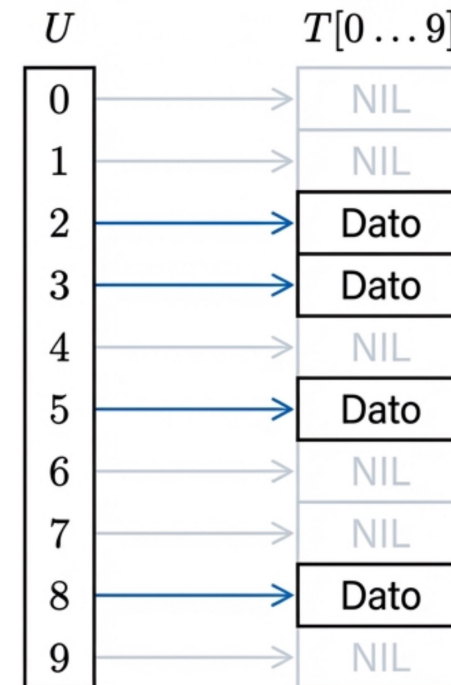
Concepto: Eficiente si el universo de claves $U = \{0, 1, \dots, m-1\}$ es pequeño.

Estructura: Un arreglo $T[0 \dots m-1]$. Cada ranura corresponde exactamente a una clave.

Operaciones $O(1)$:

- **SEARCH**(T, k) $\rightarrow T[k]$
- **INSERT**(T, x) $\rightarrow T[x.key] = x$

El Problema: Si el universo U es enorme, almacenar una tabla de tamaño $|U|$ es impráctico. Si las claves almacenadas (K) son pocas ($K \ll U$), la inmensa mayoría de la memoria se desperdicia.



Tablas hash: La función hash

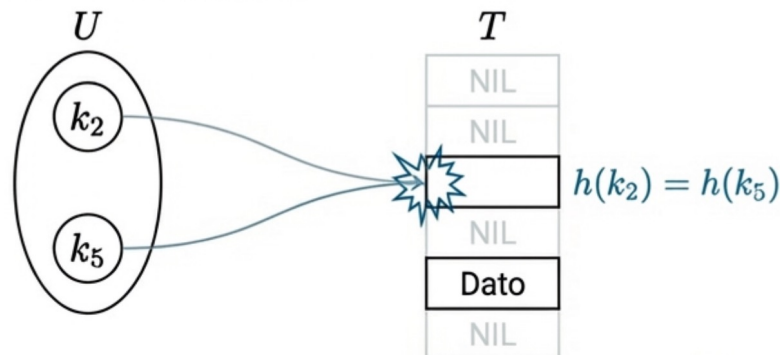
Al reducir el requerimiento de almacenamiento a $\Theta(|K|)$, una tabla hash mantiene un tiempo de búsqueda promedio de $O(1)$ sin requerir memoria para todo el universo U .

La Función Hash $h(k)$

- Calcula el índice dinámicamente en lugar de usar k directo.
- $h: U \rightarrow \{0, 1, \dots, m - 1\}$.
- El tamaño del arreglo m es mucho menor que $|U|$.

La Colisión

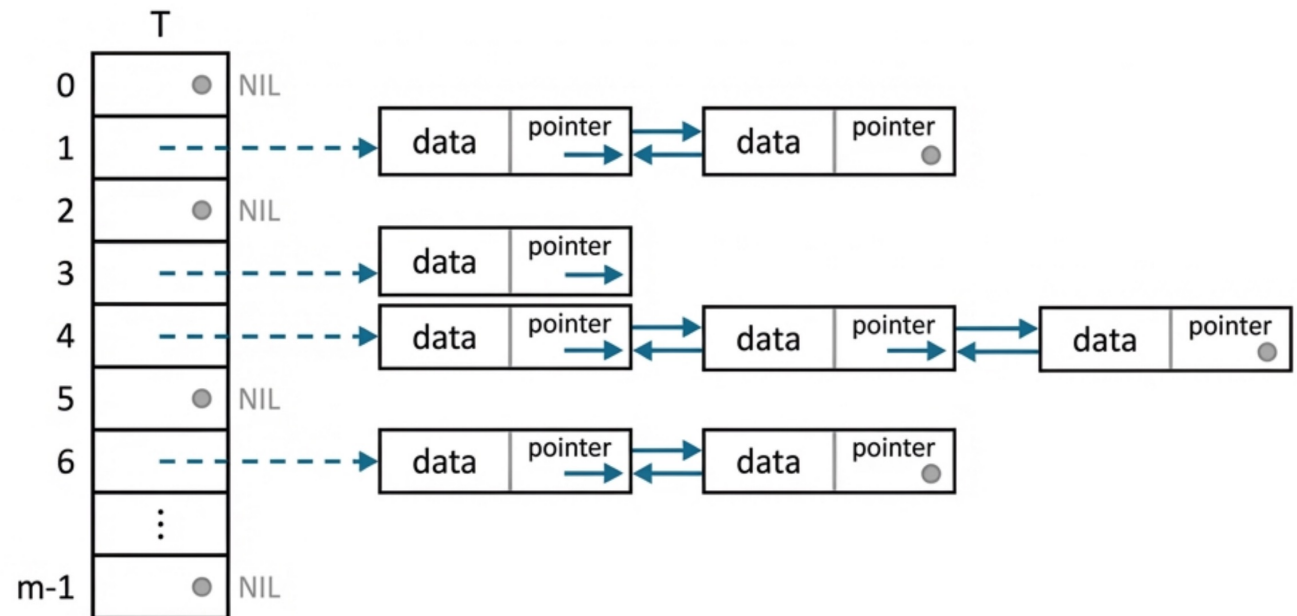
- Ocurre inexorablemente cuando $h(k_1) = h(k_2)$ para claves distintas ($k_1 \neq k_2$).



Tablas hash: Resolución de colisiones

Mecanismo: Cada ranura $T[j]$ contiene un puntero hacia una lista doblemente enlazada con todos los elementos cuya clave evalúa a j . Ranuras vacías apuntan a NIL.

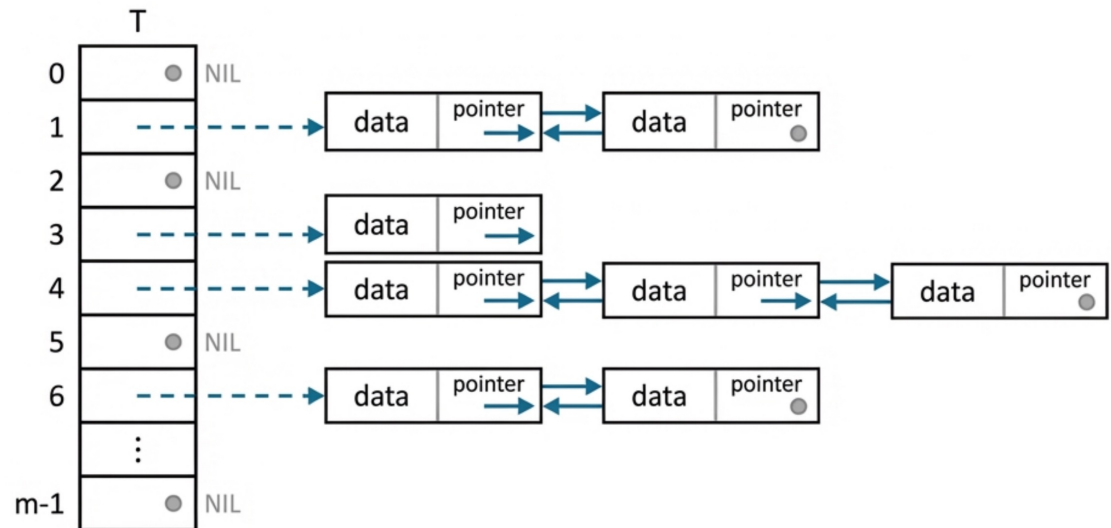
Complejidad: Inserción en $O(1)$ (en la cabeza de la lista), Eliminación en $O(1)$ (gracias al doble enlace), Búsqueda proporcional a la longitud de la cadena.



Factor de carga

Factor de Carga (α): $\alpha = \frac{n}{m}$
(n elementos en m ranuras)

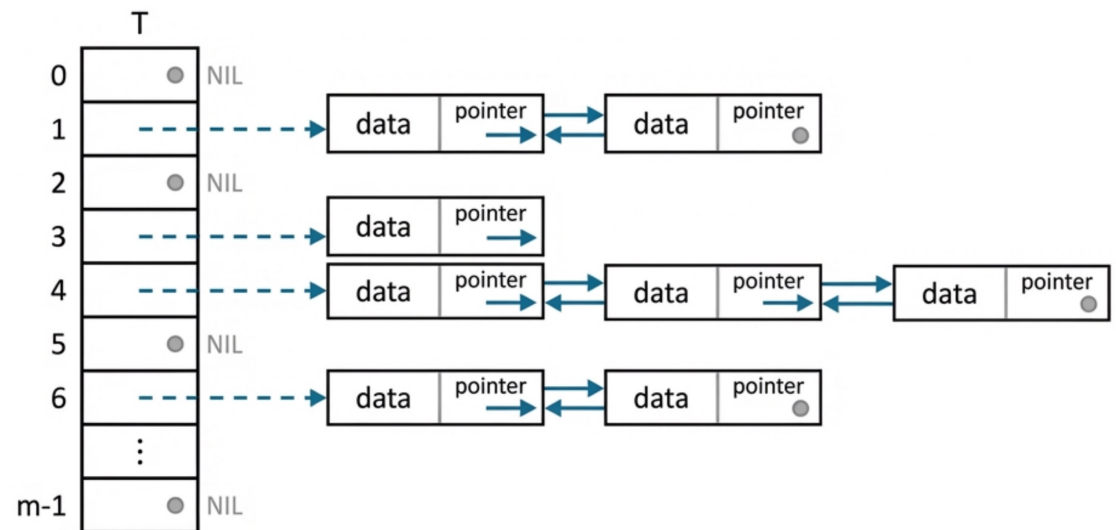
Representa el número esperado de elementos en una cadena.



Factor de carga

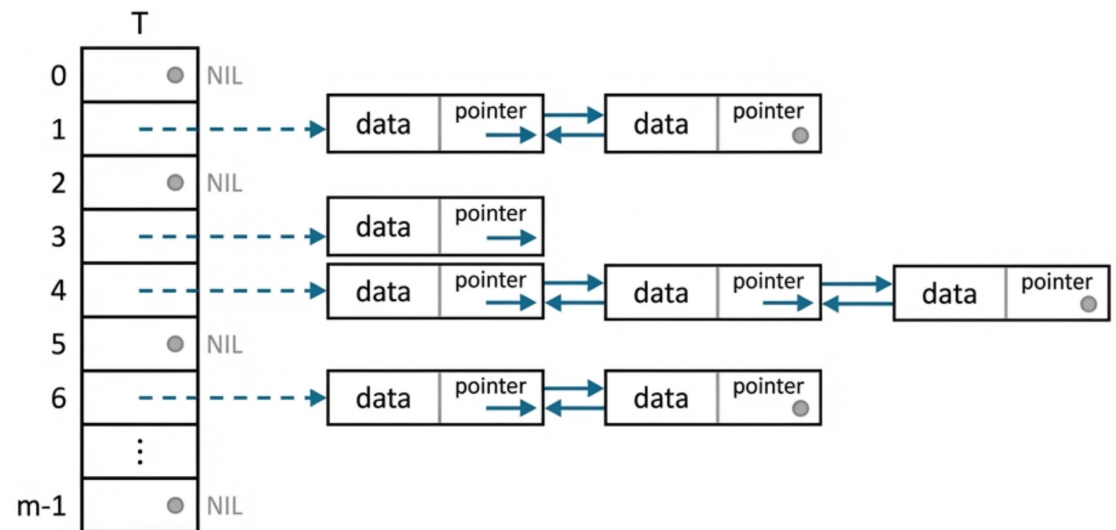
Hashing Uniforme Independiente:

Cualquier clave tiene igual probabilidad $\left(\frac{1}{m}\right)$ de caer en cualquier ranura.



Factor de carga

Búsqueda Infausta (Unsuccessful)
Explora la lista completa hasta el
final.



Funciones hash

Método de División

Fórmula: $h(k) = k \bmod m$

Ventaja: Operación única y rápida.

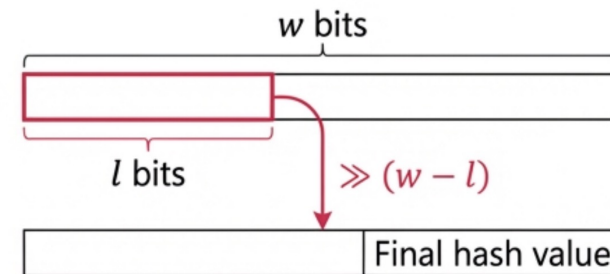
Regla de Diseño: m debe ser un número primo alejado de potencias exactas de 2 para evitar sesgos de patrones binarios.

Método de Multiplicación (Multiply-Shift)

Fórmula: $h_a(k) = (ka \bmod 2^w) \gg (w - l)$

Donde el tamaño de la tabla es $m = 2^l$.

Ventaja: Altamente eficiente en hardware.
 m puede ser una potencia exacta de 2.
Multiplica por un valor impar a ($0 < a < 2^w$).



Funciones hash

Entrada a evaluar (entero).

Tamaño de la palabra en bits
(ej. 32 o 64 bits), representa el
dominio de entrada.

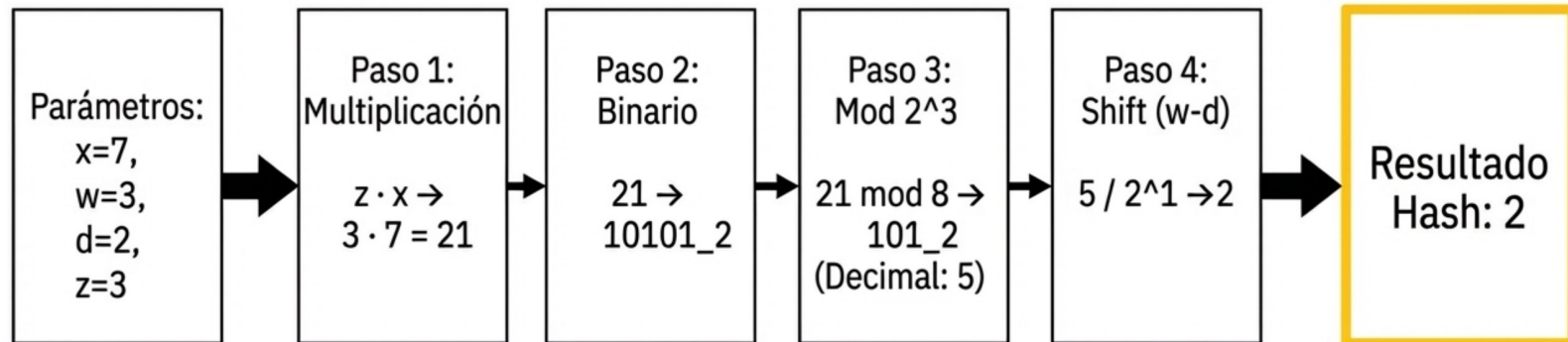
$$\text{hash}(x) = (z \cdot x) \bmod 2^w / 2^{w-d}$$

Entero impar elegido al azar
en el rango $[1, 2^w-1]$.

Número de bits que representa
el tamaño de la tabla (2^d).

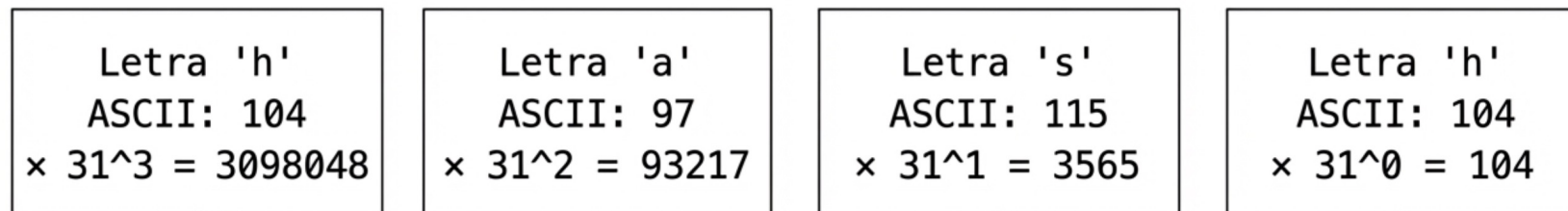
Nota: La división por 2^{w-d} se ejecuta eficientemente como un desplazamiento de bits a la derecha ($>>$).

Funciones hash



Funciones hash

$$\sum (s[i] \cdot a^{n-i-1}) \bmod m$$



Suma: 3194934
Módulo: 3194934 mod 691 (tamaño de tabla)

Valor Hash: 256

Funciones hash

Flotantes

3.14159

01000000	01001001	00001111	11011011
01000000	01001001	00001111	11011011

El Problema: Problemas de precisión y representaciones múltiples.

La Solución: Descomponer el número en su representación estricta a nivel de bits (Bitwise) antes de aplicar la función hash.

Tipos Compuestos (Ej. Punto 2D)

Estrategia: Calcular el hash individual y combinarlos polinomialmente.

Ejemplo: Point(3,4)

Asumimos: hash(3) = 83 y hash(4) = 110

$$\begin{aligned} \text{Cálculo: } & 83 \cdot 31 + 110 \\ & = 2563 + 110 \end{aligned}$$

Resultado: 2673

Funciones hash

El Problema: Para cualquier función hash estática y predecible, un atacante siempre puede elegir n claves que collisionen en la misma ranura, destruyendo el rendimiento a $\Theta(n)$.

La Solución: Aleatorización en tiempo de ejecución. Seleccionar h aleatoriamente de una familia cuidadosamente diseñada $\mathcal{H}$ independientemente de los datos de entrada.

Axioma de Universalidad

Una familia $\mathcal{H}$ es **universal** si para cualquier par de claves distintas $k_1 \neq k_2$, la probabilidad de colisión nunca es mayor que el puro azar:

$$\Pr\{h(k_1) = h(k_2)\} \leq \frac{1}{m}$$

Funciones hash

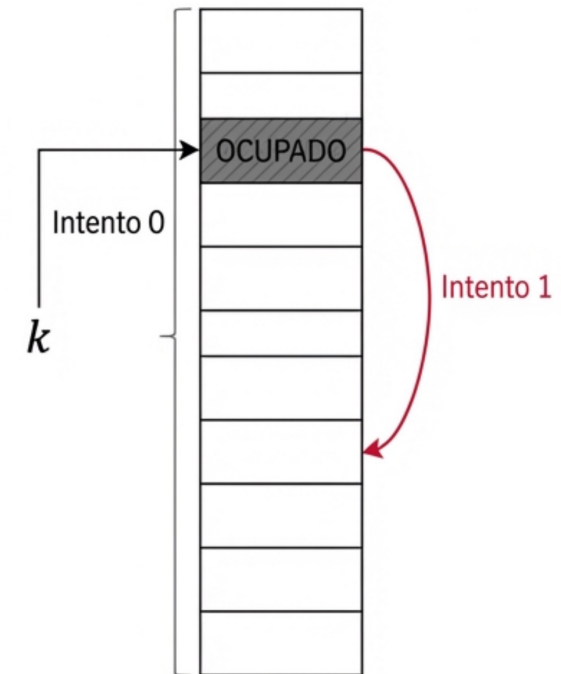
1. **Parámetros**
Elegir un número primo p donde $p > m$ y tal que toda clave $k \in \{0 \dots p-1\}$.
2. **Transformación Lineal**
Para variables aleatorias $a \in \{1 \dots p-1\}$ y $b \in \{0 \dots p-1\}$, se define:
$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$
3. **La Familia $\mathcal{H}_{pm}$**
Formada por las $p(p-1)$ funciones resultantes de la alteración de a y b . Se demuestra que esta familia es estrictamente universal.

Nota Práctica: El método de Multiply-Shift usando un multiplicador a impar aleatorio es demostrablemente $\frac{2}{m}$ -universal, siendo la variante de hardware preferida por su extrema velocidad.

Direccionamiento abierto

- **Concepto:** Eliminación total de listas enlazadas. Todos los elementos habitan exclusivamente dentro de la tabla T . Ante una colisión, se “prueba” (probe) sistemáticamente la tabla buscando la siguiente ranura vacía.
- **Regla Fundamental:** El factor de carga nunca excede la capacidad: $\alpha \leq 1$.
- **Eficiencia:** La memoria ahorrada al no instanciar punteros externos permite dimensionar tablas m más grandes para la misma huella de RAM.
- La Fórmula Ampliada:

$$h: U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$$
- La secuencia $\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$ forma una permutación completa de los índices.



Direcccionamiento abierto

Linear Probing (Prueba Lineal)

$$h(k, i) = (h_1(k) + i) \bmod m$$

Mecanismo: Si el slot está ocupado, prueba la ranura adyacente secuencialmente.

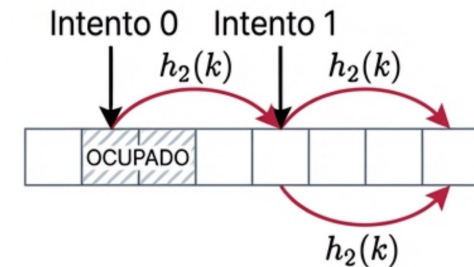
Defecto Crítico: **Clustering Primario**. Pequeñas agrupaciones de slots ocupados crecen rápidamente fusionándose, degradando severamente el tiempo de búsqueda.

Double Hashing (Hashing Doble)

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Mecanismo: El tamaño del "salto" entre intentos está dictado matemáticamente por una segunda función hash, h_2 .

Ventaja: Genera $\Theta(m^2)$ secuencias distintas. Es la mejor aproximación al hashing de permutación uniforme.



Direcccionamiento abierto

Basado en el supuesto de permutación uniforme independiente (sin eliminaciones).

Teorema: Búsqueda Infausta / Inserción

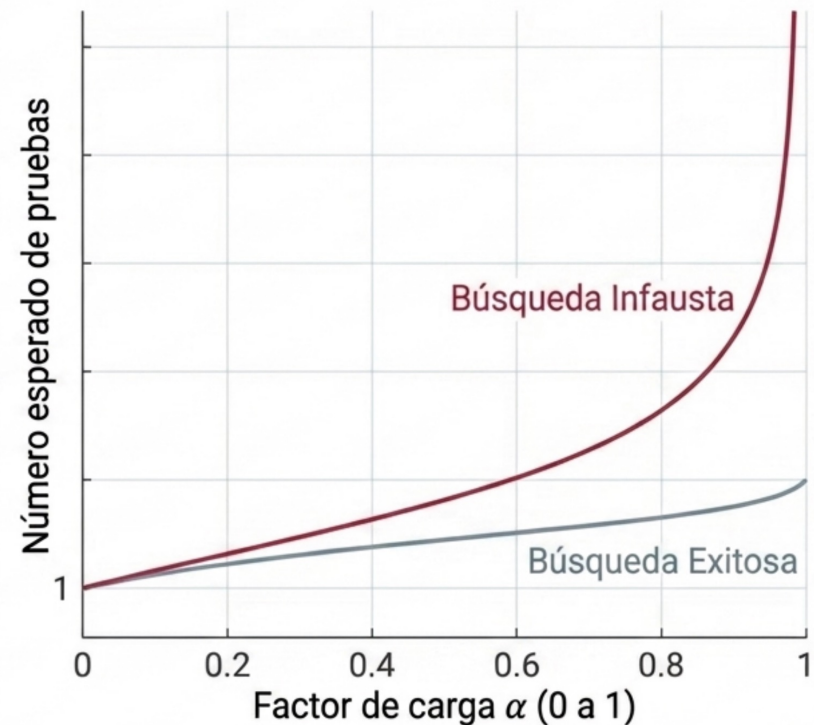
El número esperado de pruebas es $\leq \frac{1}{1 - \alpha}$

Ejemplo: Si la tabla está al 90% ($\alpha = 0.9$), tomará **~10 pruebas promedio**.

Teorema: Búsqueda Exitosa

El número esperado de pruebas es $\leq \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right)$

Ejemplo: Si $\alpha = 0.9$, encuentra elementos existentes en **< 2.559 pruebas promedio**.



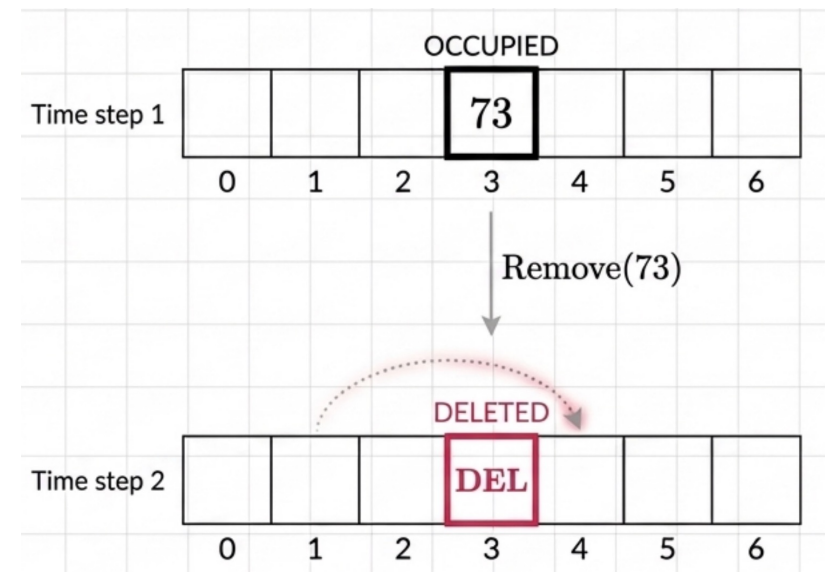
El problema

Un slot no puede simplemente vaciarse a NIL al eliminar un dato, ya que esto cortaría la secuencia de prueba de claves colisionantes previas, haciéndolas permanentemente invisibles.

La Solución Clásica (Tombstones / Marcadores):

- Reemplazar el dato con un marcador lógico especial DELETED (DEL).
- Ignora DEL y continúa avanzando.
- Sobrescribe la casilla DEL con datos nuevos.

Con altas tasas de eliminación temporal, la tabla se llena de marcadores y los tiempos de búsqueda se degradan drásticamente.



Dimensión Evaluada	Encadenamiento (Chaining)	Direccionamiento Abierto
Factor de Carga (α)	Soporta $\alpha > 1$ degradando suavemente a $O(1+\alpha)$.	Restringido estrictamente a $\alpha \leq 1$.
Uso de Punteros	Extenso overhead de memoria adicional.	Cero. Toda la memoria instanciada se usa para la tabla.
Sensibilidad al Hash	Tolerante a distribuciones pobres y colisiones en ráfaga.	Extremadamente sensible al clustering primario/secundario.
Operación DELETE	Trivial y limpia en $O(1)$ con listas dobles.	Compleja. Requiere <i>flags</i> DELETED o desplazamiento retroactivo.
Eficiencia de Caché CPU	Pobre (recorrido errático de punteros aleatorios en memoria RAM).	Excelente (alta localidad espacial, especialmente bajo <i>Linear Probing</i>).

1

La asintótica vence a los árboles.

Con $O(1)$ promedio, el hashing es el motor principal para el acceso a diccionarios dinámicos masivos, superando el límite de $\Theta(\log n)$ de los árboles de búsqueda estructurados.

2

El Hashing Universal es obligatorio.

Confiar pasivamente en que los datos poseerán una distribución uniforme es una vulnerabilidad crítica. Inyectar familias de funciones aleatorizadas asegura la defensa contra el peor caso teórico.

3

El hardware dicta la arquitectura final.

- Encadenamiento: Para diccionarios de alta volatilidad con frecuentes eliminaciones o cuando la memoria RAM no es cuello de botella.
- Dir. Abierto (Linear Probing): Apoyado por hashes criptográficos rápidos cuando el rendimiento absoluto depende de exprimir las cachés de la CPU.

